

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Scenario-Based

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Assessment Tool Weightage: 20%

Dave's Taxonomy: Manipulation (Level 2)
Question Count: 5

Total Marks: 25

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Criteria	Understanding of Scenario (1)	Identification of Risks / Issues (1)	Application of Concepts (1.25)	Decision Justification (1)	Clarity (075)	TOTAL (5)
Weightage	20%	20%	25%	20%	15%	100%
Q1						
Q2						
Q3						
Q4						
Q5						

Signature of the Faculty

Total Marks Awarded:

Scenario-Based Questions

1. A financial institution plans to deploy a private cloud with strict compliance requirements. Recommend an appropriate cloud datacenter architecture and explain how virtualization and isolation mechanisms can ensure security and regulatory compliance.
2. A SaaS provider experiences inconsistent application performance due to noisy neighbor effects in a virtualized environment. Analyze the root cause and propose resource allocation and hypervisor-level solutions.
3. An organization wants to migrate legacy monolithic applications into a Service-Oriented Architecture (SOA) in the cloud. Explain the migration strategy and how SOA improves scalability and maintainability.
4. A government agency is implementing identity federation across multiple departments using cloud services. Explain how identity management, authentication, and privacy policies should be designed to ensure secure access.
5. A cloud provider wants to implement real-time user presence detection for collaborative applications. Discuss how presence services can be integrated into cloud architecture while maintaining user privacy and system scalability.

1. A Financial Institution Plans to Deploy a Private Cloud with Strict Compliance Requirements

Introduction

Financial institutions such as banks, insurance companies, and investment firms manage highly confidential customer information, including account details, financial transactions, credit card information, loan records, and personal identification data. Any unauthorized access or data breach can result in financial losses, legal penalties, and loss of customer trust. Therefore, a **Private Cloud Datacenter Architecture** is the most suitable deployment model because it provides dedicated infrastructure, greater administrative control, enhanced security, and compliance with regulatory standards such as **PCI-DSS, ISO 27001, GDPR, HIPAA (where applicable), and RBI guidelines**. The architecture must be designed to ensure confidentiality, integrity, availability, and accountability of data.

Recommended Cloud Datacenter Architecture

1. Physical Infrastructure Layer

The physical infrastructure forms the foundation of the private cloud and includes enterprise-grade servers, storage devices, networking equipment, backup power supplies, cooling systems, and redundant internet connections. Redundant hardware ensures that services remain available even if a component fails, thereby improving business continuity.

2. Virtualization Layer

A hypervisor such as VMware ESXi, Microsoft Hyper-V, or KVM creates multiple Virtual Machines (VMs) on a single physical server. Each VM operates independently with its own operating system and applications. Virtualization improves hardware utilization, reduces operational costs, enables quick deployment of services, and simplifies disaster recovery through features such as snapshots and live migration.

3. Cloud Management Layer

Cloud management software such as OpenStack or VMware vCloud manages the provisioning, monitoring, automation, and orchestration of virtual resources. It enables administrators to allocate computing resources dynamically, monitor system health, enforce policies, and automate routine management tasks.

4. Security Layer

The security layer includes firewalls, Intrusion Detection Systems (IDS), Intrusion Prevention Systems (IPS), anti-malware solutions, encryption mechanisms, and Identity and Access Management (IAM). These components protect the infrastructure against cyber threats, unauthorized access, malware infections, and insider attacks.

5. Application Layer

The application layer hosts banking applications, online transaction systems, customer relationship management (CRM) software, analytics platforms, and reporting systems.

Applications should communicate securely through encrypted channels and follow secure coding practices.

Role of Virtualization in Security

Virtualization contributes significantly to cloud security by logically separating workloads running on shared hardware. Each virtual machine operates in an isolated environment, reducing the possibility of interference between applications. Virtualization also supports rapid provisioning, fault isolation, disaster recovery, and efficient resource utilization.

Isolation Mechanisms

Strong isolation mechanisms are essential to protect sensitive financial information. These include virtual machine isolation, network segmentation using VLANs, dedicated virtual storage volumes, secure hypervisor configurations, CPU and memory reservations, and virtual firewalls. Isolation prevents one compromised VM from affecting other workloads running on the same physical server.

Conclusion

A well-designed private cloud architecture combined with virtualization, strong isolation techniques, encryption, access control, and continuous monitoring provides financial institutions with a secure, reliable, and compliant computing environment. It protects sensitive customer information while ensuring uninterrupted financial services and regulatory compliance.

2. SaaS Provider Experiences Inconsistent Performance Due to Noisy Neighbor Effects

Introduction

In a cloud environment, multiple virtual machines often share the same physical resources. When one VM consumes an excessive amount of CPU, memory, storage, or network bandwidth, the performance of neighbouring VMs degrades. This issue is known as the **Noisy Neighbor Problem**. It can negatively impact application performance, customer satisfaction, and Service Level Agreement (SLA) compliance.

Root Causes

CPU Contention

When several VMs compete for limited CPU resources, processing delays occur. High-priority workloads may experience increased response times due to CPU scheduling conflicts.

Memory Contention

If one VM consumes excessive RAM, the hypervisor may perform memory ballooning or swapping, significantly reducing application performance.

Storage I/O Bottlenecks

Multiple VMs performing simultaneous read/write operations on shared storage devices can increase disk latency and reduce throughput.

Network Congestion

Heavy network traffic generated by one VM can reduce available bandwidth for other VMs, causing packet delays and slower application responses.

Overcommitment of Resources

Allocating more virtual resources than physically available increases resource contention and performance instability.

Resource Allocation Solutions

- CPU reservations guarantee processing power for critical workloads.
- Memory reservations ensure dedicated RAM.
- Resource pools prioritize business-critical applications.
- Storage QoS limits excessive disk usage.
- Network QoS prioritizes important traffic.
- Load balancing distributes workloads evenly.
- Auto-scaling automatically provisions additional resources during peak demand.
- Capacity planning prevents excessive resource overcommitment.

Hypervisor-Level Solutions

Hypervisors provide advanced scheduling algorithms, CPU affinity settings, NUMA optimization, live VM migration, storage optimization, memory compression, and transparent page sharing. These mechanisms reduce resource contention and improve workload isolation.

Monitoring and Performance Analysis

Administrators should continuously monitor CPU utilization, memory consumption, storage latency, network bandwidth, and application response times using tools such as VMware vCenter, Prometheus, Grafana, Nagios, or Microsoft System Center.

Conclusion

The noisy neighbour problem can be minimized through intelligent resource allocation, Quality of Service (QoS), load balancing, hypervisor optimization, and continuous monitoring, resulting in a stable and high-performing SaaS environment.

3. An Organization Wants to Migrate Legacy Monolithic Applications into a Service-Oriented Architecture (SOA) in the Cloud. Explain the Migration Strategy and How SOA Improves Scalability and Maintainability.

Introduction

Many organizations still use **legacy monolithic applications**, where all business functions such as user interface, business logic, and database access are combined into a single

application. Although monolithic systems are simple to develop initially, they become difficult to maintain as the organization grows. Updating one module often requires redeploying the entire application, leading to increased downtime and maintenance costs. Service-Oriented Architecture (SOA) is a software architecture that divides a large application into smaller, independent, and reusable services. Each service performs a specific business function and communicates with other services through standard protocols such as REST, SOAP, HTTP, or XML. Migrating legacy applications to SOA enables organizations to achieve greater flexibility, scalability, maintainability, and easier cloud integration.

Migration Strategy

1. Assessment of the Existing Application

The first step is to perform a comprehensive analysis of the existing monolithic application. Developers identify all business processes, software modules, databases, external interfaces, and system dependencies. This assessment helps determine which components should be converted into independent services.

2. Identify Business Services

The application is divided into separate business services based on functionality. For example, an e-commerce application can be separated into user management, product catalogue, payment processing, order management, and inventory management services. Each service becomes independent and reusable.

3. Design Standard Service Interfaces

Each service should expose its functionality through standard APIs such as RESTful APIs or SOAP web services. Standardized interfaces allow services developed using different programming languages or running on different platforms to communicate seamlessly.

4. Database Modernization

Legacy applications often use a single centralized database. During migration, the database can be redesigned so that each service manages its own data or securely shares data through APIs. This reduces dependencies and improves performance.

5. Deploy Services on Cloud Infrastructure

Each service is deployed independently on virtual machines, containers, or cloud platforms. Cloud environments provide elasticity, allowing services to scale automatically based on workload demands.

6. Testing and Validation

Comprehensive testing is essential after migration. Functional testing verifies service correctness, integration testing ensures communication between services, performance testing evaluates scalability, and security testing identifies vulnerabilities.

7. Continuous Monitoring and Maintenance

Cloud monitoring tools continuously track service availability, CPU utilization, memory consumption, response times, and failures. Automatic alerts help administrators resolve issues before they affect users.

How SOA Improves Scalability

SOA significantly improves scalability because each service can be scaled independently without affecting the rest of the application.

For example:

- If payment requests increase during online shopping festivals, only the payment service requires additional resources.
- The inventory service continues operating normally without consuming unnecessary computing resources.
- Load balancing distributes requests among multiple service instances.
- Cloud auto-scaling automatically increases or decreases resources based on demand.

Independent scaling reduces infrastructure costs while maintaining high application performance.

How SOA Improves Maintainability

SOA simplifies application maintenance through modular design.

Benefits include:

- Developers modify only the affected service without redeploying the entire application.
- Software bugs are easier to locate because services are isolated.
- New features can be introduced with minimal disruption.
- Independent development teams can work simultaneously on different services.
- Software updates become faster and less risky.

This modular approach significantly reduces maintenance effort and operational downtime.

Conclusion

Migrating legacy monolithic applications to Service-Oriented Architecture enables organizations to modernize their software systems while improving scalability, maintainability, flexibility, and reliability. Although migration requires careful planning, testing, and monitoring, the long-term benefits include lower maintenance costs, faster development, better cloud integration, and improved business agility.

4. A Government Agency is Implementing Identity Federation Across Multiple Departments Using Cloud Services. Explain How

Identity Management, Authentication, and Privacy Policies Should Be Designed to Ensure Secure Access.

Introduction

Government organizations manage highly confidential information, including citizen records, tax information, healthcare data, land records, and law enforcement databases. Employees often require access to services across multiple departments. Identity Federation allows users to access multiple cloud services using a single trusted digital identity without creating separate accounts for every department.

A secure identity federation system should combine Identity and Access Management (IAM), strong authentication, authorization mechanisms, and comprehensive privacy policies.

Identity Management (IAM)

Identity Management is responsible for creating, maintaining, updating, and deleting digital identities throughout the user lifecycle.

The IAM system should include:

User Registration

Every employee should receive a unique digital identity linked to official government records.

Identity Verification

Identity verification should involve document validation, employee verification, biometric confirmation, or digital certificates before access is granted.

Identity Lifecycle Management

When employees join, transfer, retire, or leave government service, their permissions should automatically be updated or revoked.

Role Management

Employees should receive permissions based on their job responsibilities rather than individual requests.

Authentication

Authentication verifies that users are genuinely who they claim to be.

Recommended authentication methods include:

Multi-Factor Authentication (MFA)

Users authenticate using multiple verification methods, such as:

- Password
- One-Time Password (OTP)
- Fingerprint recognition
- Face recognition

- Smart cards
- Security tokens

Using multiple authentication factors significantly reduces the risk of unauthorized access.

Single Sign-On (SSO)

Single Sign-On allows employees to authenticate once and securely access multiple departmental applications without repeatedly entering usernames and passwords.

Benefits include:

- Improved user convenience
- Reduced password fatigue
- Better security management
- Simplified administration

Authorization

Authorization determines what resources authenticated users are permitted to access.

Recommended authorization models include:

Role-Based Access Control (RBAC)

Permissions are assigned according to employee roles.

Examples:

- Police officers access criminal databases.
- Doctors access healthcare records.
- Revenue officers access taxation systems.

Attribute-Based Access Control (ABAC)

Access decisions are based on user attributes such as:

- Department
- Designation
- Security clearance
- Time of access
- Geographic location

ABAC provides more flexible and context-aware security than RBAC.

Identity Federation Standards

Government agencies should implement internationally accepted standards:

- Security Assertion Markup Language (SAML)
- OAuth 2.0
- OpenID Connect (OIDC)

These standards enable secure authentication and authorization between departments while maintaining interoperability.

Privacy Policies

Privacy policies should protect citizens' sensitive information through:

- Data minimization
- Explicit user consent where applicable
- Encryption of stored and transmitted data
- Audit trails
- Secure backup procedures
- Data retention policies
- Compliance with national data protection regulations

Only authorized personnel should access confidential information.

Conclusion

Identity federation enables secure collaboration between multiple government departments while maintaining strong security and privacy. By combining IAM, MFA, SSO, RBAC, ABAC, encryption, audit logging, and Zero Trust principles, government agencies can provide efficient, secure, and compliant access to cloud services.

5. A Cloud Provider Wants to Implement Real-Time User Presence Detection for Collaborative Applications. Discuss How Presence Services Can Be Integrated into Cloud Architecture While Maintaining User Privacy and System Scalability.

Introduction

Real-time presence services allow users to see the availability status of others, such as **Online, Offline, Busy, Away, or Do Not Disturb**. These services are widely used in messaging applications, video conferencing platforms, online learning systems, collaborative workspaces, customer support systems, and social networking applications. A cloud-based presence service must support millions of simultaneous users while ensuring fast updates, privacy protection, and high availability.

Cloud Architecture for Presence Services

A scalable architecture consists of:

Client Layer

Users connect through web browsers, desktop applications, or mobile devices.

Load Balancer

Incoming requests are distributed evenly across multiple presence servers, preventing server overload.

Presence Servers

Presence servers receive user status updates, maintain active sessions, and distribute changes to connected users.

Message Broker

Technologies such as Apache Kafka or RabbitMQ manage event distribution efficiently between services.

Distributed Database

Presence information is stored in distributed databases to provide high availability and fault tolerance.

Cache Layer

Redis or Memcached stores frequently accessed presence information, reducing database load and improving response time.

Monitoring Layer

Monitoring tools continuously observe system health, latency, resource utilization, and failures.

Working Process

1. User logs into the application.
2. Authentication verifies user identity.
3. Presence server records the user's status.
4. Status information is stored in the distributed database.
5. The message broker broadcasts updates to subscribed users.
6. Connected clients receive instant notifications through WebSockets or similar real-time communication technologies.
7. The cache serves frequently requested presence information to reduce database access.

Maintaining User Privacy

Privacy is essential because presence information reveals user activity.

Important privacy measures include:

User-Controlled Privacy Settings

Users should decide:

- Who can view their status.
- Whether they appear online or offline.
- Which contacts receive presence updates.

Encryption

All communication should use TLS/HTTPS, while stored presence data should be encrypted to prevent unauthorized access.

Authentication and Authorization

Only authenticated users should access presence services. RBAC ensures users view only authorized information.

Data Minimization

Only essential presence information should be stored and shared.

Compliance

The system should comply with relevant privacy regulations such as GDPR or local data protection laws.

Ensuring Scalability

To support millions of users, the architecture should implement:

Horizontal Scaling

Additional presence servers can be added during peak demand.

Load Balancing

Requests are distributed evenly across multiple servers to avoid bottlenecks.

Distributed Databases

Data replication ensures high availability and fault tolerance.

Auto Scaling

Cloud resources automatically increase or decrease according to workload.

Caching

Frequently accessed information is stored in Redis to reduce latency.

Event-Driven Architecture

Status updates are transmitted only when changes occur, reducing unnecessary network traffic.

WebSockets

Persistent WebSocket connections enable instant bidirectional communication between clients and servers.

Conclusion

Integrating presence services into cloud architecture requires a combination of scalable infrastructure, efficient communication protocols, strong security, and privacy protection. Technologies such as microservices, load balancing, distributed databases, caching, WebSockets, encryption, and continuous monitoring enable cloud providers to deliver reliable, secure, and highly scalable real-time collaboration services that meet the demands of modern users.